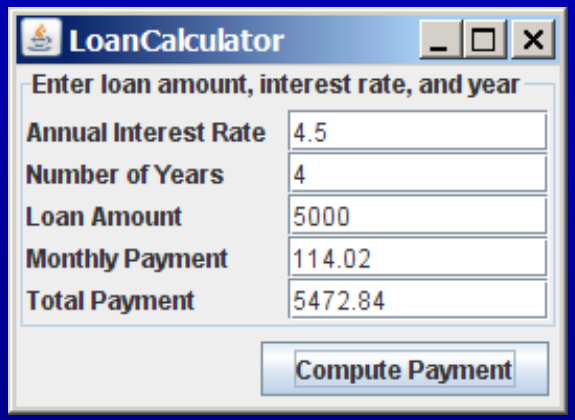
Bölüm 16: Olaya-Dayalı Programlama (Event-Driven Programming)



Motivasyon

Kullanıcının kredi tutarını, yıllık faiz oranını ve yıl sayısını girmesini sağlayan ve hesapla butonuna bastığınızda aylık ödeme ile toplam ödemeyi gösteren bir GUI programı yazmak istediğinizi varsayalım. Bu programı nasıl yazarsınız? Buton tıklatma olayına yanıt vermek üzere kodu yazmak için olaya-dayalı programlama kullanmanız gerekir.



LoanCalculator

Run

Motivations

Şekil'de gösterildiği gibi, yükselen bir bayrağı canlandıran bir program yazmak istediğinizi varsayalım. Bunu nasıl başarırsınız? Bu sorunun birkaç çözümü vardır. Bunu çözmenin etkili bir yolu, bu bölümün konusu olan olaya-dayalı programlamada bir zamanlayıcı kullanmaktır.



Amaçlar

- Olayları, olay kaynaklarını ve olay sınıflarını tanımlamak(§16.2).
- Dinleyici sınıflarını tanımlamak için, dinleyici nesnelerini kaynak nesneyle kaydedin ve olayları işleyecek kodu yazın (§16.3).
- İç sınıfları kullanarak dinleyici sınıflarını tanımlayabilme(§16.4).
- Anonim iç sınıflar kullanarak dinleyici sınıflarını tanımlamak(§16.5).
- Dinleyici oluşturmak ve kaydetmek için çeşitli kodlama stillerini keşfetmek (§16.6).
- Bir düğmeyi tıklattığınızda metin alanından girdi almak için (§16.7).
- WindowEvent ile başa çıkmak için programlar yazmak(§16.8).
- Dinleyici arayüzü bağdaştırıcılarını kullanarak dinleyici sınıfları için kodlamayı basitleştirmek için (§16.9).
- MouseEvent ile başa çıkmak için programlar yazmak(§16.10).
- KeyEvent ile başa çıkmak için programlar yazmak(§16.11).
- Animasyonları kontrol etmek için javax.swing.Timer sınıfı kullanma (§16.12).

Prosedürel ve Olaya Dayalı Programlama

- *Prosedürel programlama* işlem sırasına göre yürütülür..
- *Olaya-Dayalı Programlama*da ise kod, olayların etkinleştirilmesi üzerine yürütülür.

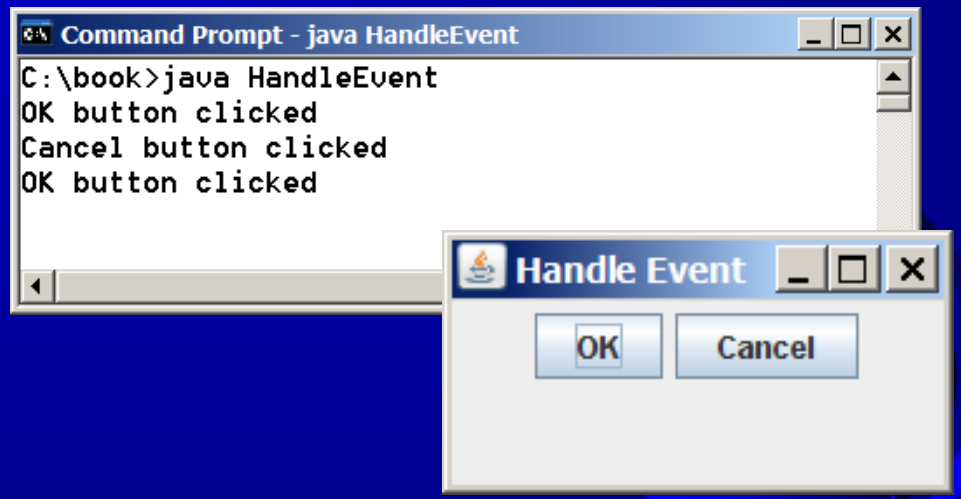


Revisit Listing 11.7 Olaya-Dayalı Programlama'nın güzel yanı

- Örnek, çerçevede bir düğme görüntüler. Bir düğme tıklandığında konsolda bir mesaj görüntülenir

HandleEvent

Run



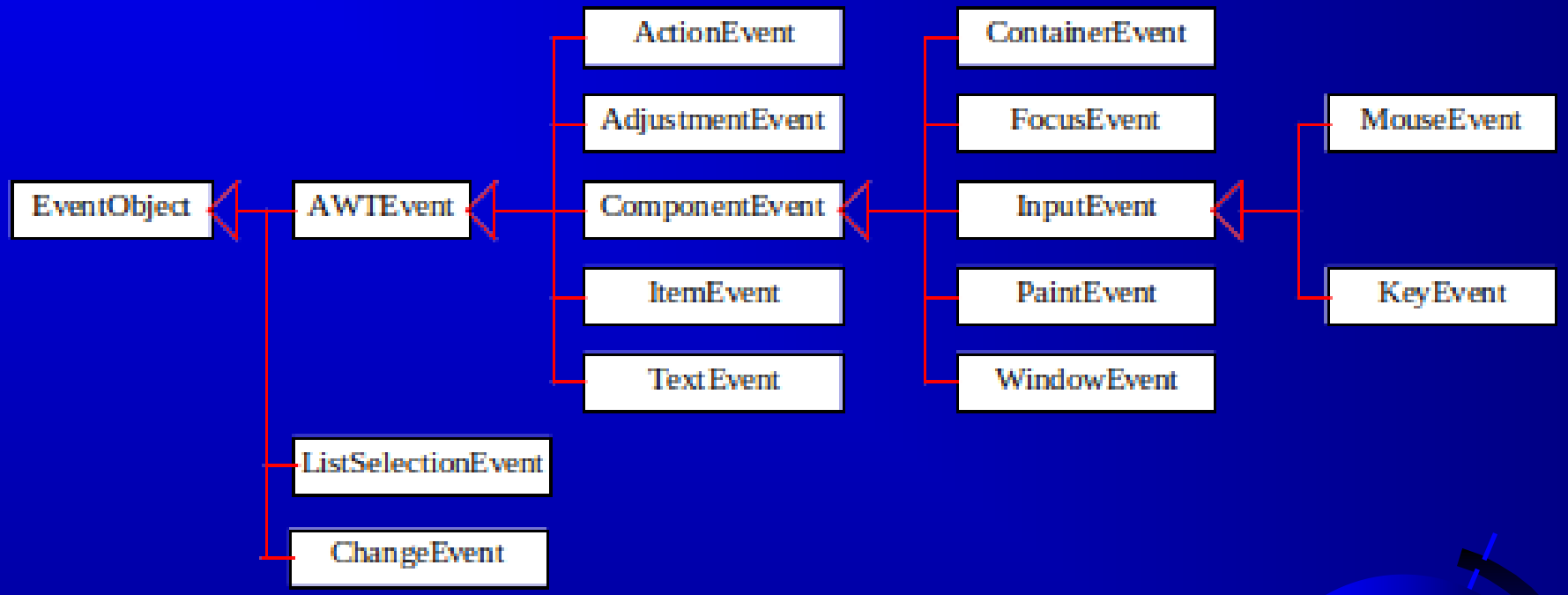
Olaylar (Events)

- Bir olay, programa bir şeyin gerçekleştiğine dair bir sinyal türü olarak tanımlanabilir.

- Olay, fare hareketleri, fare tıklamaları ve tuş vuruşları gibi harici kullanıcı eylemleri tarafından veya zamanlayıcı gibi işletim sistemi tarafından oluşturulur.



Olay Sınıfları (Event Classes)



Olay Bilgisi (Event Information)

Bir olay nesnesi, olayla ilgili olan özellikleri içerir. EventObject sınıfındaki getSource () örnek yöntemini kullanarak olayın kaynak nesnesini tanımlayabilirsiniz. EventObject alt sınıfları, düğme eylemleri, pencere olayları, bileşen olayları, fare hareketleri ve tuş vuruşları gibi özel olay türleriyle ilgilenir. Tablo 15.1, harici kullanıcı eylemlerini, kaynak nesneleri ve oluşturulan olay türlerini listeler.

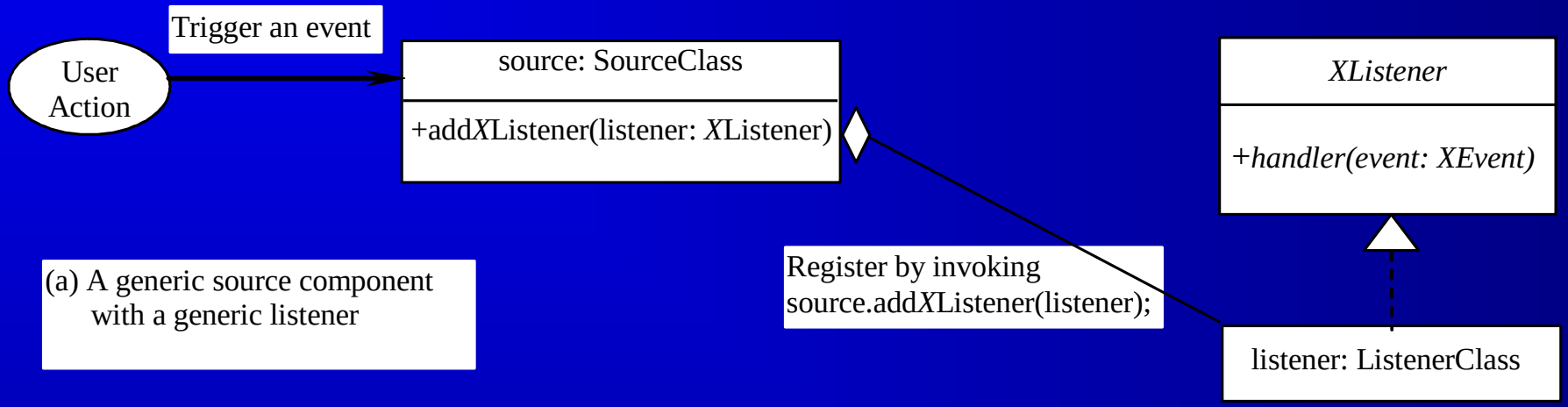


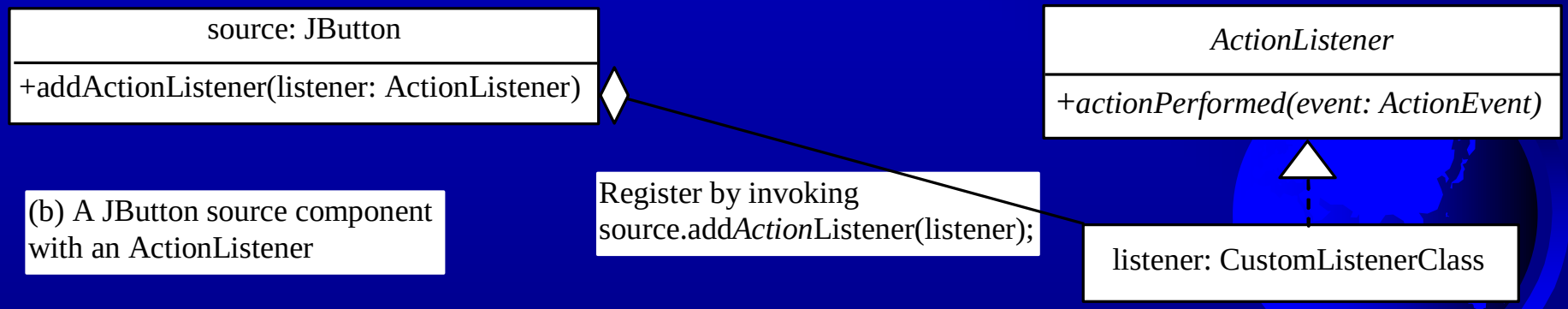
Seçili Kullanıcı Eylemleri

Source User Action	Event Type Object	Generated
Click a button	`JButton`	`ActionEvent`
Click a check box	`JCheckBox`	`ItemEvent`, `ActionEvent`
Click a radio button	`JRadioButton`	`ItemEvent`, `ActionEvent`
Press return on a text field	`JTextField`	`ActionEvent`
Select a new item	`JComboBox`	`ItemEvent`, `ActionEvent`
Window opened, closed, etc.	`Window`	`WindowEvent`
Mouse pressed, released, etc.	`Component`	`MouseEvent`
Key released, pressed, etc.	`Component`	`KeyEvent`

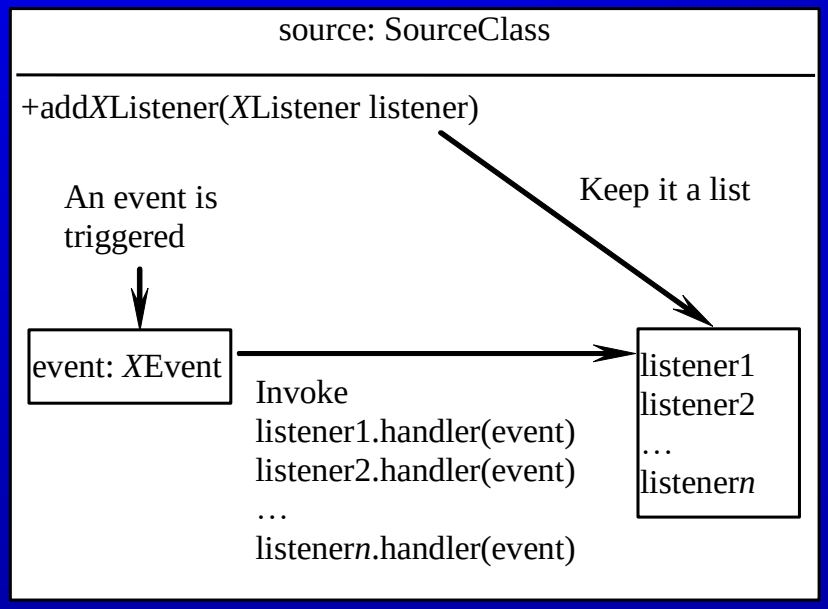


Yetkilendirme Modeli

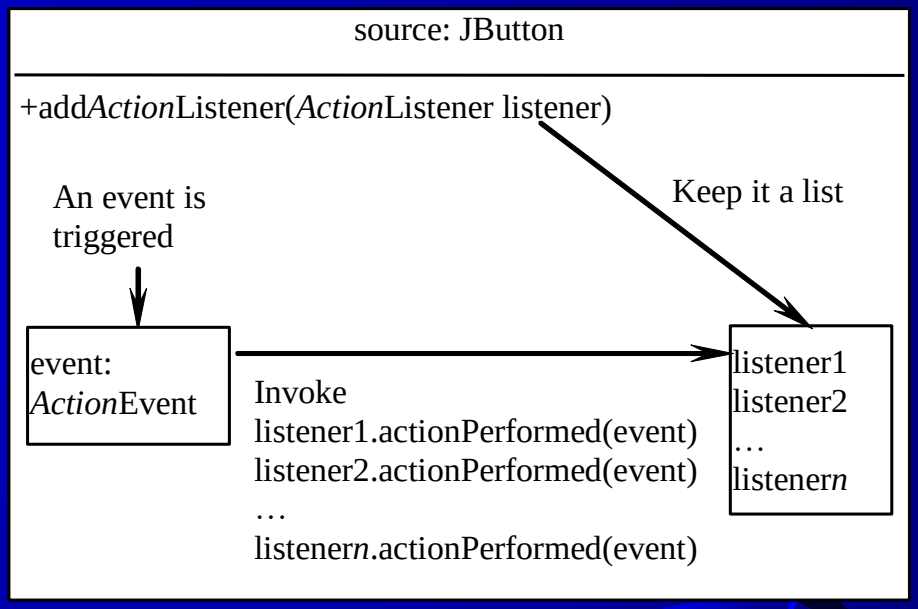




Kaynak Bileşenin Dahili İşlevi



(a) Internal function of a generic source object



(b) Internal function of a JButton object

Yetkilendirme Modeli: Örnek

```
JButton jbt = new JButton("OK");

ActionListener listener = new OKListener();

jbt.addActionListener(listener);
```

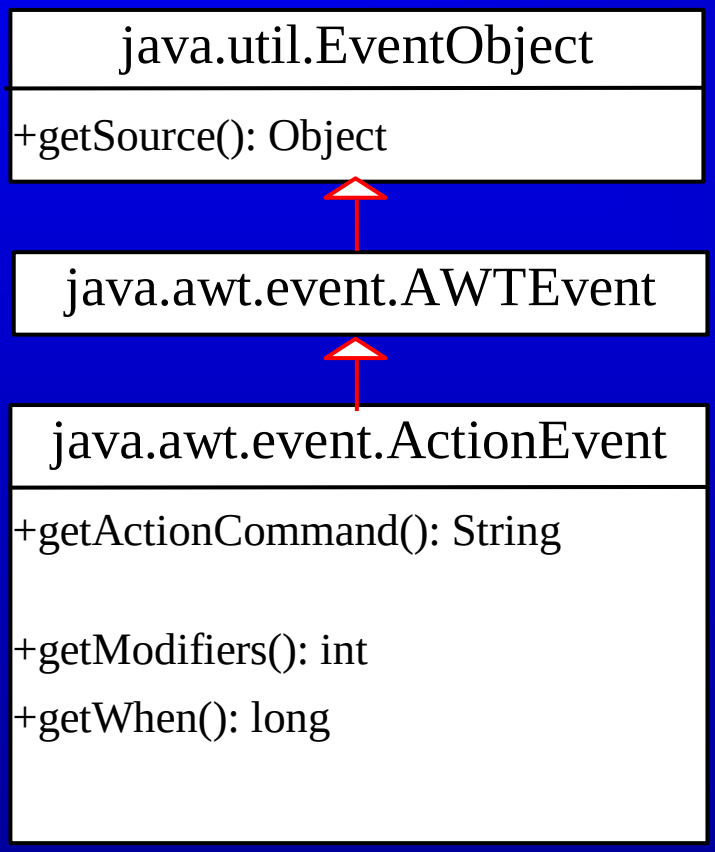


Seçili Olay İşleyicileri

Event Class	Listener Interface	Listener Methods (Handlers)
ActionEvent	ActionListener	actionPerformed(ActionEvent)
ItemEvent	ItemListener	itemStateChanged(ItemEvent)
WindowEvent	WindowListener	windowClosing(WindowEvent)
		windowOpened(WindowEvent)
		windowIconified(WindowEvent)
		windowDeiconified(WindowEvent)
		windowClosed(WindowEvent)
		windowActivated(WindowEvent)
		windowDeactivated(WindowEvent)
ContainerEvent	ContainerListener	componentAdded(ContainerEvent)
		componentRemoved(ContainerEvent)
MouseEvent		
MouseListener	mousePressed(MouseEvent)	
	mouseReleased(MouseEvent)	
		mouseClicked(MouseEvent)
		mouseExited(MouseEvent)
		mouseEntered(MouseEvent)
KeyEvent	KeyListener	keyPressed(KeyEvent)
	keyReleased(KeyEvent)	
		keyTypeed(KeyEvent)



java.awt.event.ActionEvent



Returns the object on which the event initially occurred.

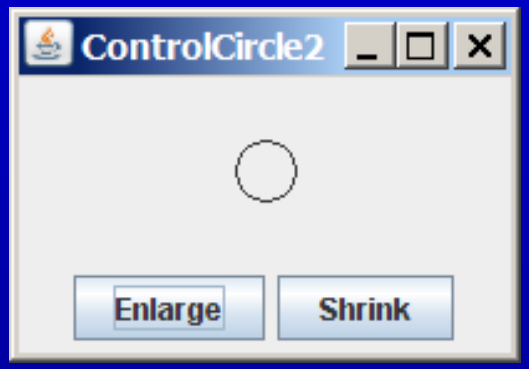
Returns the command string associated with this action. For a button, its text is the command string.

Returns the modifier keys held down during this action event.

Returns the timestamp when this event occurred. The time is the number of milliseconds since January 1, 1970, 00:00:00 GMT.

Örnek: ControlCircle için ilk sürüm (no listeners)

Şimdi bir dairenin boyutunu kontrol etmek için iki düğme kullanan bir program yazmayı düşünelim.



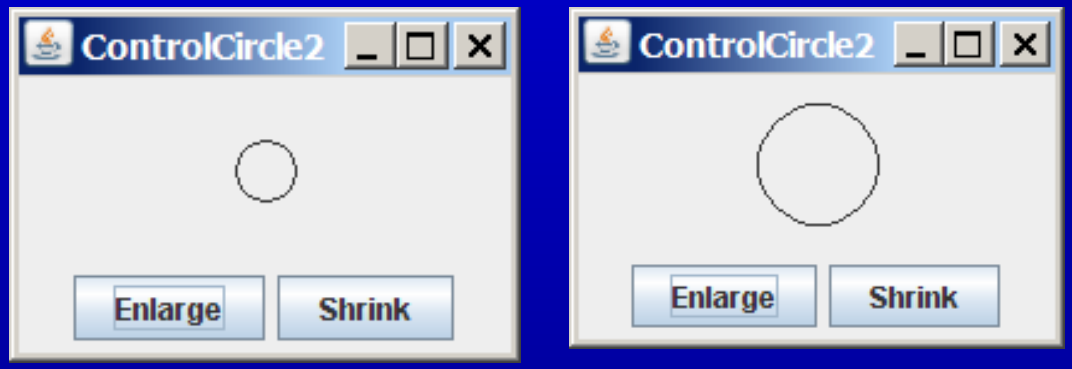


ControlCircle1

Run

Örnek: ControlCircle için İkinci sürüm (with listener for Enlarge)

Şimdi bir dairenin boyutunu kontrol etmek için iki düğme kullanan bir program yazmayı düşünelim.



ControlCircle2

Run

Dahili sınıf Dinleyicileri (Inner Class Listeners)

Bir dinleyici sınıfı, özel olarak bir GUI bileşeni (örn. Bir düğme) için bir dinleyici nesnesi oluşturmak üzere tasarlanmıştır. Başka uygulamalar tarafından paylaşılmayacak. Bu nedenle, frame sınıfı içindeki dinleyici sınıfını bir dahili sınıf olarak tanımlamak uygundur.



Dahili Sınıflar (Inner Classes)

Inner class: Başka bir sınıfın üyesi olan sınıftır. A

Avantajları: Bazı uygulamalarda, programları basitleştirmek için bir dahili sınıf kullanabilirsiniz.

- Bir dahili sınıf, içinde bulunduğu dış sınıfta tanımlanan verilere ve yöntemlere başvurabilir, bu nedenle dış sınıfın referansını iç sınıfın yapıcısına iletmeniz gerekmez.

ShowInnerClass



Dahili Sınıflar

```
public class Test {
  ...
}

public class A {
  ...
}
```

(a)

```
public class Test {
  ...

  // Inner class
  public class A {
    ...
  }
}
```

(b)

```
// OuterClass.java: inner class demo
public class OuterClass {
  private int data;

  /** A method in the outer class */
  public void m() {
    // Do something
  }

  // An inner class
  class InnerClass {
    /** A method in the inner class */
    public void mi() {
      // Directly reference data and method
      // defined in its outer class
      data++;
      m();
    }
  }
}
```

(c)

Dahili Sınıflar ...

- Dahili sınıflar programları basit, az ve öz hale getirebilir.
- Dahili sınıf, içerdiği dış sınıfın çalışmasını destekler ve *OuterClassName*$*InnerClassName*.class adlı bir sınıfa derlenir. Örneğin, OuterClass içindeki InnerClass *OuterClass*$*InnerClass*.class içine derlenir .



Dahili Sınıflar ...

- Bir dahili sınıf, sınıfın bir üyesine uygulanan aynı görünürlük kurallarına tabi olarak public, protected, veya private olarak ilan edilebilir.
- Dahili sınıf static olarak ilan edilebilir. Bir static dahili sınıfına dış sınıf ismi kullanılarak erişilebilir. Statik bir dahili sınıf, dış sınıfın static olmayan üyelerine erişemez



Anonim Dahili Sınıflar

- Anonim bir dahili sınıf her zaman bir üst sınıfı genişletmeli veya bir arayüz uygulamalıdır, ancak açık bir genişletme veya uygulama yan tümcesi olamaz.
- Anonim bir dahili sınıf, tüm soyut yöntemleri üst sınıfta veya arayüzde uygulamalıdır
- Anonim bir dahili sınıf, bir örnek oluşturmak için her zaman üst sınıfındaki boş yapıcısını kullanır. Anonim bir dahili sınıf bir arayüz uygularsa, yapıcı Object () olur.
- Anonim bir dahili sınıf, OuterClassName $ n.class adlı bir sınıfa derlenir. Örneğin, dış sınıf Test'in iki anonim iç sınıfı varsa, bu iki sınıf Test $ 1.class ve Test $ 2.class olarak derlenir.

Anonim Dahili Sınıflar

Dahili sınıf dinleyicileri anonim dahili sınıflar kullanılarak kısaltılabilir. Anonim bir dahili sınıf, adı olmayan bir dahili sınıftır. Bir dahili sınıfı bildirmeyi ve sınıfın bir örneğini bir adımda oluşturmayı birleştirir. Anonim bir dahili sınıf şöyle bildirilir:

```
new SuperClassName/InterfaceName() {
  // Implement or override methods in superclass or interface
  // Other methods if necessary
}
```

AnonymousListenerDemo

Run

Dinleyici Sınıflarını Tanımlamanın Alternatif Yolları

Dinleyici sınıflarını tanımlamanın başka birçok yolu vardır. Örneğin, yalnızca bir dinleyici oluşturarak Liste dinleyicisini yeniden yazabilir, dinleyiciyi düğmeleriyle kaydedebilir ve dinleyicinin olay kaynağını, yani hangi düğmenin etkinliği tetiklediğini algılamasına izin verebilirsiniz.T

DetectSourceDemo

Run

Dinleyici Sınıflarını Tanımlamanın Alternatif Yolları

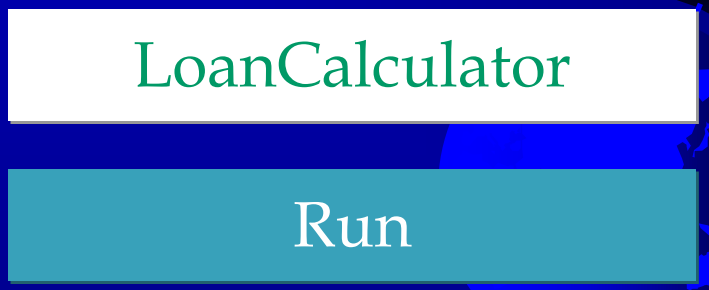
ActionListener uygulayan özel çerçeve sınıfını da tanımlayabilirsiniz.



FrameAsListenerDemo

Run

Problem: Loan Calculator



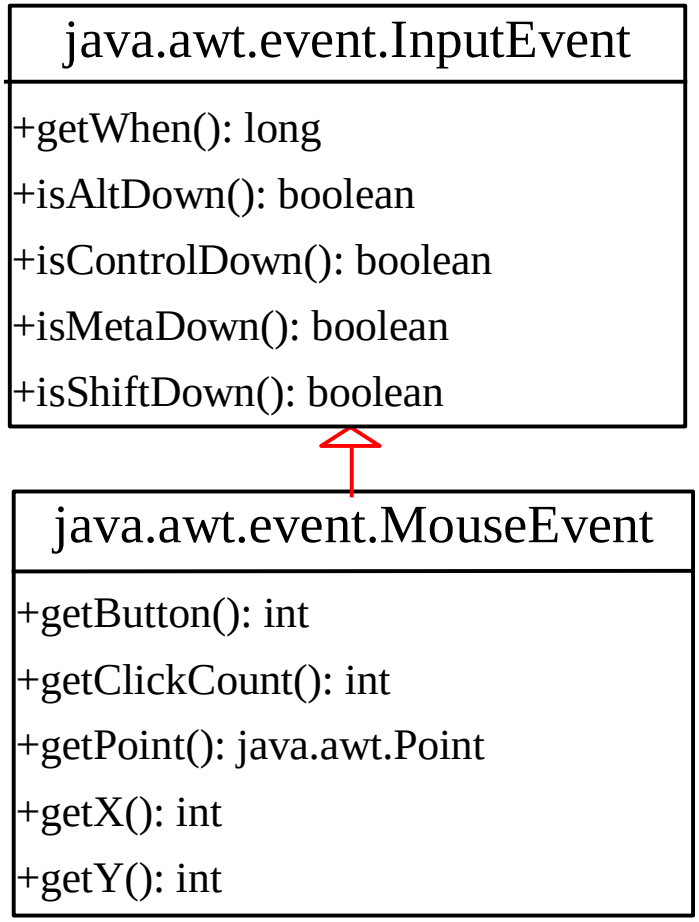
Örnek: Pencere Olaylarını İşleme

Amaç: Pencere olaylarını ele almayı gösterme. Window sınıfının herhangi bir alt sınıfı aşağıdaki pencere olaylarını oluşturabilir: pencere açılır, kapanır, kapatılır, etkinleştirilir, devre dışı bırakılır, simgeselleştirilir ve dekonifiye edilir. Bu program bir çerçeve oluşturur, pencere olaylarını dinler ve gerçekleşen olayı belirtmek için bir mesaj görüntüler.

TestWindowEvent

Run

MouseEvent



Returns the timestamp when this event occurred.

Returns whether or not the Alt modifier is down on this event.

Returns whether or not the Control modifier is down on this event.

Returns whether or not the Meta modifier is down on this event

Returns whether or not the Shift modifier is down on this event.

Indicates which mouse button has been clicked.

Returns the number of mouse clicks associated with this event.

Returns a Point object containing the x and y coordinates.

Returns the x-coordinate of the mouse point.

Returns the y-coordinate of the mouse point.

Fare Olaylarını İşleme

- Java Fare olaylarını işlemek için, `MouseListener` ve `MouseMotionListener` olarak iki tane dinleyici arayüzü sağlar.
- *MouseListener*, fareye basıldığı, bırakıldığı, girildiği, çıkıldığı veya tıklandığı gibi eylemleri dinler.
- *MouseMotionListener* fare sürükleme veya taşıma gibi eylemleri dinler.



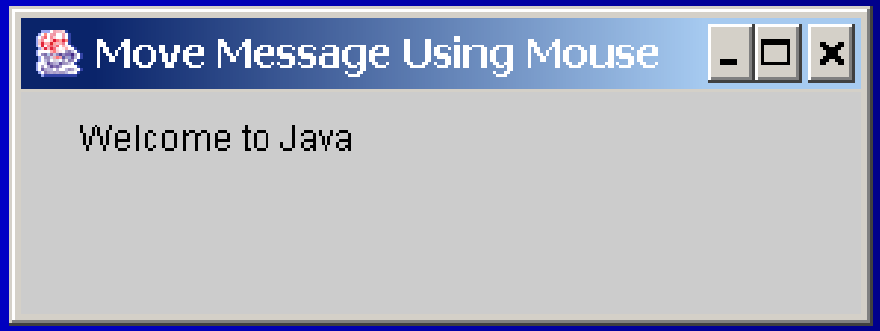
Handling Mouse Events

java.awt.event.MouseListener	
+mousePressed(e: MouseEvent): void	Invoked when the mouse button has been pressed on the source component.
+mouseReleased(e: MouseEvent): void	Invoked when the mouse button has been released on the source component.
+mouseClicked(e: MouseEvent): void	Invoked when the mouse button has been clicked (pressed and released) on the source component.
+mouseEntered(e: MouseEvent): void	Invoked when the mouse enters the source component.
+mouseExited(e: MouseEvent): void	Invoked when the mouse exits the source component.

java.awt.event.MouseMotionListener	
+mouseDragged(e: MouseEvent): void	Invoked when a mouse button is moved with a button pressed.
+mouseMoved(e: MouseEvent): void	Invoked when a mouse button is moved without a button pressed.

Örnek: Fareyi Kullanarak İletiyi Taşıma

Amaç: İletiyi panelde görüntülemek için bir program oluşturun. Mesajı taşımak için fareyi kullanabilirsiniz. Fare sürüklendikçe mesaj hareket eder ve her zaman fare noktasında görüntülenir.



MoveMessageDemo

Run

Klavye Olaylarını İşleme

Klavye olaylarını işlemek için `KeyListener` arayüzünde aşağıdaki işleyiciler kullanılır: :

- `keyPressed(KeyEvent e)`

Bir tuşa basıldığında çağrılır.

- `keyReleased(KeyEvent e)`

Bir tuş bırakıldığında çağrılır.

- `keyTyped(KeyEvent e)`

Bir tuşa basıldıktan sonra bırakıldığında çağrılır.



KeyEvent Sınıfı

- Methods:

  `getKeyChar()` method

  `getKeyCode()` method

- Keys:

  Home `VK_HOME`

  End `VK_END`

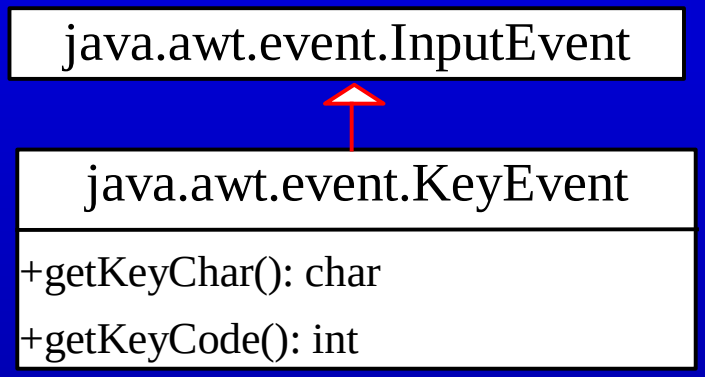
  Page Up `VK_PGUP`

  Page Down `VK_PGDN`

  etc...



KeyEvent Sınıfı...

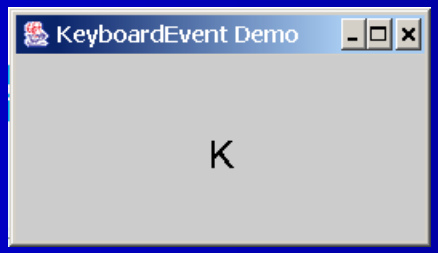


Returns the character associated with the key in this event.

Returns the integer keyCode associated with the key in this event.

Örnek: Klavye Olayları Demosu

Amaç: Bir kullanıcı giriş karakteri görüntülemek. Kullanıcı ayrıca ok tuşlarını kullanarak karakteri yukarı, aşağı, sola ve sağa hareket ettirebilir.



KeyEventDemo

Run

Timer Sınıfı

GUI olmayan bazı bileşenler olayları tetikleyebilir. Javax.swing.Timer sınıfı, bir ActionEvent'i önceden tanımlanmış bir oranda tetikleyen bir kaynak bileşendir.

javax.swing.Timer	
+Timer(delay: int, listener: ActionListener)	Creates a Timer with a specified delay in milliseconds and an ActionListener.
+addActionListener(listener: ActionListener): void	Adds an ActionListener to the timer.
+start(): void	Starts this timer.
+stop(): void	Stops this timer.
+setDelay(delay: int): void	Sets a new delay value for this timer.

Timer sınıfı animasyonları kontrol etmek için kullanılabilir. Örneğin, hareketli bir mesajı görüntülemek için kullanabilirsiniz.

AnimationDemo

Run

Saat Animasyonu

Bölüm 14'te, geçerli saati göstermek için bir StillClock çizdiniz. Saat görüntülendikten sonra işaretlemiyor. Saatin her saniye yeni bir saat göstermesini sağlamak için ne yapabilirsiniz? Saati işaretlemenin anahtarı, her saniye yeni bir geçerli saatle yeniden boyamaktır. Saatin nasıl yeniden boyanacağını kontrol etmek için bir zamanlayıcı kullanabilirsiniz.

ClockAnimation

Run